Introduction to Robotics

Lecture 7: Velocity Control and Introduction to Trajectory Generation

Konstantinos Chatzilygeroudis - costashatz@upatras.gr

Department of Electrical and Computer Engineering
University of Patras

Template made by Panagiotis Papagiannopoulos

Joint Control with Velocities (1)

- We assume that we have a desired trajetory $\boldsymbol{q}_d(t)$
- For example, $\boldsymbol{q}_d(t) = \alpha sin(t)$
- The easiest way to control the joint is to give velocity commands:

$$\dot{\boldsymbol{q}}(t) = \dot{\boldsymbol{q}}_d(t) = \alpha cos(t)$$

- **Feedforward** or **open-loop** controller
- There is not *feedback* from sensors!

Joint Control with Velocities (2)

- How can we exploit sensor readings? In other words, how can we do **feedback control**?

Joint Control with Velocities (2)

- How can we exploit sensor readings? In other words, how can we do **feedback control**?
- **P-controller:** $\dot{\boldsymbol{q}}(t) = \boldsymbol{K}_p(\boldsymbol{q}_d(t) - \boldsymbol{q}(t))$, $\boldsymbol{K}_p > 0$
- $\boldsymbol{q}_e(t) = (\boldsymbol{q}_d(t) - \boldsymbol{q}(t))$
- **PI-controller:** $\dot{\boldsymbol{q}}(t) = \boldsymbol{K}_p \boldsymbol{q}_e(t) + \boldsymbol{K}_i \int_0^t \boldsymbol{q}_e(t)dt$, $\boldsymbol{K}_p, \boldsymbol{K}_i > 0$
- If $\dot{\boldsymbol{q}}_d(t) = constant$, then the PI-controller removes the steady state error.

Joint Control with Velocities (3)

- Feedback control needs an error signal to "begin"!

Joint Control with Velocities (3)

- Feedback control needs an error signal to "begin"!
- Let's combine the open-loop and the feedback control loops:

$$\dot{\boldsymbol{q}}(t) = \dot{\boldsymbol{q}}_d(t) + \boldsymbol{K}_p \boldsymbol{q}_e(t) + \boldsymbol{K}_i \int_0^t \boldsymbol{q}_e(t) dt$$

$$\boldsymbol{K}_p, \boldsymbol{K}_i > 0$$

- **Feedforward Plus Feedback Control**

What if we have multiple joints?

What if we have multiple joints?

- We assume that every joint is "independent"
- Thus we have n controllers (where n is the number of joints)
- $\boldsymbol{K}_p \in \mathbb{R}^{n \times n}$
- $\boldsymbol{K}_i \in \mathbb{R}^{n \times n}$
- $\boldsymbol{K}_p, \boldsymbol{K}_i$ usually have values only in the diagonal
- Otherwise, there is correlation between joints

Single Joint Velocity Control - Code Example

jupyter 1-joint-control Last Checkpoint: 35 seconds ago

File Edit View Run Kernel Settings Help

Code JupyterLab Python 3

[ ]:
```python
import numpy as np # Linear Algebra
import matplotlib.pyplot as plt # Plotting
```

[ ]:
```python
# Simplest possible system
static = True
limits = False

Kp = 2.
Ki = 0.1

qd = 5.
def desired(t):
    if static:
        return qd # static
    return qd * np.cos(t)

def desired_v(t):
```

Velocity Control in Task-Space

If we have a velocity profile $\mathcal{V}_d(t)$ for the end-effector?

Velocity Control in Task-Space

If we have a velocity profile $\mathcal{V}_d(t)$ for the end-effector?

- $\mathcal{V}_b(t) = [Ad_{\boldsymbol{T}_{bw}}]\mathcal{V}_d(t) + K_p\mathcal{X}_e(t) + K_i\int_0^t \mathcal{X}_e(t)dt, K_p, K_i > 0$
- $[\mathcal{X}_e] = \log(\boldsymbol{T}_{wb}^{-1}\boldsymbol{T}_{wd})$
- $\mathcal{V}_b = \boldsymbol{J}_b\boldsymbol{v} \implies \boldsymbol{v}(t) = \boldsymbol{J}_b^{\dagger}(\boldsymbol{q})\mathcal{V}_b(t)$
- $\boldsymbol{v} = \boldsymbol{J}_b^{\dagger}\mathcal{V}_b$

Velocity Control in Task-Space (2)

Pseudoinverse can be unstable around singularities, in practice we use:

- $\boldsymbol{v} = \alpha \boldsymbol{J}^T \mathcal{V}, \alpha \in \mathbb{R}^+$

- Damped Pseudoinverse:
  - $\boldsymbol{J}^\dagger = \boldsymbol{J}^T(\boldsymbol{J}\boldsymbol{J}^T + \lambda^2 \boldsymbol{I})^{-1}$, if $n > m, (\boldsymbol{J}^\dagger \boldsymbol{J} = \boldsymbol{I})$
  - $\boldsymbol{J}^\dagger = (\boldsymbol{J}^T\boldsymbol{J} + \lambda^2 \boldsymbol{I})^{-1}\boldsymbol{J}^T$, if $n < m, (\boldsymbol{J}\boldsymbol{J}^\dagger = \boldsymbol{I})$
  - $\lambda \in \mathbb{R}^+$

- We can use optimization-based control!

Task-Space Control - Code Example

jupyter 2-tasks-space-control Last Checkpoint: 17 minutes ago

File Edit View Run Kernel Settings Help

Code Open in...

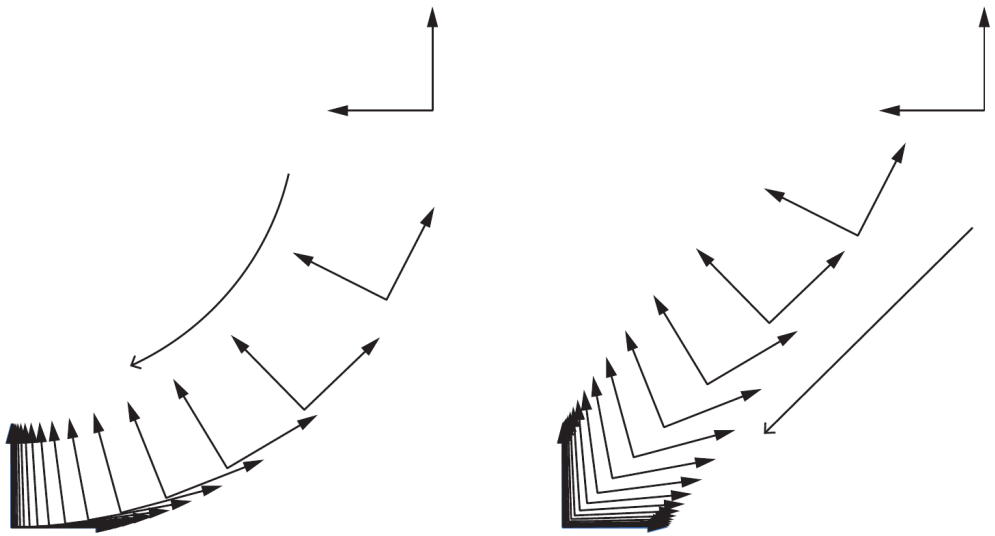
[ ]:
```python
import numpy as np # Linear Algebra
import matplotlib.pyplot as plt # Plotting
import pinocchio as pin # dynamics/simulation

from pinocchio.robot_wrapper import RobotWrapper
from pinocchio.shortcuts import buildModelsFromUrdf
from pinocchio.visualize import MeshcatVisualizer

VISUALIZER = MeshcatVisualizer
```

[ ]:
```python
# Pinocchio related
urdf_file = "/usr/share/utheque/iiwa/iiwa.urdf"
```

Separating Position and Orientation



Source: Modern Robotics: Mechanics, Planning, and Control, *Kevin M. Lynch and Frank C. Park*, 2017, Cambridge University Press.

Trajectory Generation

- How can we generate a full trajectory (with timings) given an initial, $\boldsymbol{x}_s$ and target, $\boldsymbol{x}_g$, state?
- Let's do the simplest one! *Straight line:*

Trajectory Generation

- How can we generate a full trajectory (with timings) given an initial, $\boldsymbol{x}_s$ and target, $\boldsymbol{x}_g$, state?
- Let's do the simplest one! *Straight line:*

$$\boldsymbol{x}(t) = \boldsymbol{x}_s + t(\boldsymbol{x}_g - \boldsymbol{x}_s)$$

- What is the issue with this?

Trajectory Generation

- How can we generate a full trajectory (with timings) given an initial, $\boldsymbol{x}_s$ and target, $\boldsymbol{x}_g$, state?
- Let's do the simplest one! *Straight line:*

$$\boldsymbol{x}(t) = \boldsymbol{x}_s + t(\boldsymbol{x}_g - \boldsymbol{x}_s)$$

- What is the issue with this? What if $t > 1$?
- We also need to specify the final time T:

$$\boldsymbol{x}(t) = \boldsymbol{x}_s + \frac{t}{T}(\boldsymbol{x}_g - \boldsymbol{x}_s)$$

Trajectory Generation

- How can we generate a full trajectory (with timings) given an initial, $\boldsymbol{x}_s$ and target, $\boldsymbol{x}_g$, state?
- Let's do the simplest one! *Straight line:*

$$\boldsymbol{x}(t) = \boldsymbol{x}_s + t(\boldsymbol{x}_g - \boldsymbol{x}_s)$$

- What is the issue with this? What if $t > 1$?
- We also need to specify the final time T:

$$\boldsymbol{x}(t) = \boldsymbol{x}_s + \frac{t}{T}(\boldsymbol{x}_g - \boldsymbol{x}_s)$$

- What if $t > T$ or if I do not start from $t = 0$?

$$\boldsymbol{x}(t) = \boldsymbol{x}_s + \max(1, \frac{t - t_0}{T})(\boldsymbol{x}_g - \boldsymbol{x}_s)$$

Trajectory Generation (2)

- What if $\boldsymbol{x}$ contains velocities? Aka, $\boldsymbol{x} = \begin{bmatrix} \boldsymbol{q} \\ \dot{\boldsymbol{q}} \end{bmatrix}$
- Can we use the previous?

Trajectory Generation (2)

- What if $\boldsymbol{x}$ contains velocities? Aka, $\boldsymbol{x} = \begin{bmatrix} \boldsymbol{q} \\ \dot{\boldsymbol{q}} \end{bmatrix}$
- Can we use the previous? **We can only have constant velocity!** Why?

Trajectory Generation (2)

- What if $\boldsymbol{x}$ contains velocities? Aka, $\boldsymbol{x} = \begin{bmatrix} \boldsymbol{q} \\ \dot{\boldsymbol{q}} \end{bmatrix}$
- Can we use the previous? **We can only have constant velocity!** Why?
- Using the above, we model the path as $\boldsymbol{q}(t) = \boldsymbol{c}_1 t + \boldsymbol{c}_0$. Thus $\dot{\boldsymbol{q}}(t) = \boldsymbol{c}_1$!!! What can we do?

Trajectory Generation (2)

- What if $\boldsymbol{x}$ contains velocities? Aka, $\boldsymbol{x} = \begin{bmatrix} \boldsymbol{q} \\ \dot{\boldsymbol{q}} \end{bmatrix}$
- Can we use the previous? **We can only have constant velocity!** Why?
- Using the above, we model the path as $\boldsymbol{q}(t) = \boldsymbol{c}_1 t + \boldsymbol{c}_0$. Thus $\dot{\boldsymbol{q}}(t) = \boldsymbol{c}_1$!!! What can we do?
- Let's use higher order polynomial: $\boldsymbol{q}(t) = \boldsymbol{c}_2 t^2 + \boldsymbol{c}_1 t + \boldsymbol{c}_0$
- Now $\dot{\boldsymbol{q}}(t) = 2\boldsymbol{c}_2 t + \boldsymbol{c}_1$
- Much better!

Trajectory Generation (3)

$$\boldsymbol{q}(t) = \boldsymbol{c}_2 t^2 + \boldsymbol{c}_1 t + \boldsymbol{c}_0$$
$$\dot{\boldsymbol{q}}(t) = 2\boldsymbol{c}_2 t + \boldsymbol{c}_1$$

- What now? How can we compute the $\boldsymbol{c}_i$s?
- Given $\boldsymbol{x}_s = \begin{bmatrix} \boldsymbol{q}_s \\ \dot{\boldsymbol{q}}_s \end{bmatrix}$, $\boldsymbol{x}_g = \begin{bmatrix} \boldsymbol{q}_g \\ \dot{\boldsymbol{q}}_g \end{bmatrix}$ and a total time T:

Trajectory Generation (3)

$$\boldsymbol{q}(t) = \boldsymbol{c}_2 t^2 + \boldsymbol{c}_1 t + \boldsymbol{c}_0$$
$$\dot{\boldsymbol{q}}(t) = 2\boldsymbol{c}_2 t + \boldsymbol{c}_1$$

- What now? How can we compute the $\boldsymbol{c}_i$s?
- Given $\boldsymbol{x}_s = \begin{bmatrix} \boldsymbol{q}_s \\ \dot{\boldsymbol{q}}_s \end{bmatrix}$, $\boldsymbol{x}_g = \begin{bmatrix} \boldsymbol{q}_g \\ \dot{\boldsymbol{q}}_g \end{bmatrix}$ and a total time T:

$$\boldsymbol{q}(0) = \boldsymbol{q}_s = \boldsymbol{c}_0$$
$$\dot{\boldsymbol{q}}(0) = \dot{\boldsymbol{q}}_s = \boldsymbol{c}_1$$

- Nice! Let's find the last coefficient!

Trajectory Generation (3)

$$\boldsymbol{q}(t) = \boldsymbol{c}_2 t^2 + \boldsymbol{c}_1 t + \boldsymbol{c}_0$$
$$\dot{\boldsymbol{q}}(t) = 2\boldsymbol{c}_2 t + \boldsymbol{c}_1$$

- What now? How can we compute the $\boldsymbol{c}_i$s?
- Given $\boldsymbol{x}_s = \begin{bmatrix} \boldsymbol{q}_s \\ \dot{\boldsymbol{q}}_s \end{bmatrix}$, $\boldsymbol{x}_g = \begin{bmatrix} \boldsymbol{q}_g \\ \dot{\boldsymbol{q}}_g \end{bmatrix}$ and a total time T:

$$\boldsymbol{q}(0) = \boldsymbol{q}_s = \boldsymbol{c}_0$$
$$\dot{\boldsymbol{q}}(0) = \dot{\boldsymbol{q}}_s = \boldsymbol{c}_1$$

- Nice! Let's find the last coefficient!

$$\begin{aligned} \boldsymbol{q}(T) = \boldsymbol{q}_g &= \boldsymbol{c}_2 T^2 + \boldsymbol{c}_1 T + \boldsymbol{c}_0 \\ &= \boldsymbol{c}_2 T^2 + \dot{\boldsymbol{q}}_s T + \boldsymbol{q}_s \Rightarrow \boldsymbol{c}_2 = \frac{\boldsymbol{q}_g - \dot{\boldsymbol{q}}_s T - \boldsymbol{q}_s}{T^2} \end{aligned}$$

Trajectory Generation (3)

$$\boldsymbol{q}(t) = \boldsymbol{c}_2 t^2 + \boldsymbol{c}_1 t + \boldsymbol{c}_0$$
$$\dot{\boldsymbol{q}}(t) = 2\boldsymbol{c}_2 t + \boldsymbol{c}_1$$

- What now? How can we compute the $\boldsymbol{c}_i$s?
- Given $\boldsymbol{x}_s = \begin{bmatrix} \boldsymbol{q}_s \\ \dot{\boldsymbol{q}}_s \end{bmatrix}$, $\boldsymbol{x}_g = \begin{bmatrix} \boldsymbol{q}_g \\ \dot{\boldsymbol{q}}_g \end{bmatrix}$ and a total time T:

$$\boldsymbol{q}(0) = \boldsymbol{q}_s = \boldsymbol{c}_0$$
$$\dot{\boldsymbol{q}}(0) = \dot{\boldsymbol{q}}_s = \boldsymbol{c}_1$$

- Nice! Let's find the last coefficient!

$$\boldsymbol{q}(T) = \boldsymbol{q}_g = \boldsymbol{c}_2 T^2 + \boldsymbol{c}_1 T + \boldsymbol{c}_0$$
$$= \boldsymbol{c}_2 T^2 + \dot{\boldsymbol{q}}_s T + \boldsymbol{q}_s \Rightarrow \boldsymbol{c}_2 = \frac{\boldsymbol{q}_g - \dot{\boldsymbol{q}}_s T - \boldsymbol{q}_s}{T^2}$$

- OR

$$\dot{\boldsymbol{q}}(T) = \dot{\boldsymbol{q}}_g = 2\boldsymbol{c}_2 T + \dot{\boldsymbol{q}}_s \Rightarrow \boldsymbol{c}_2 = \frac{\dot{\boldsymbol{q}}_g - \dot{\boldsymbol{q}}_s}{2T}$$

Cubic Splines

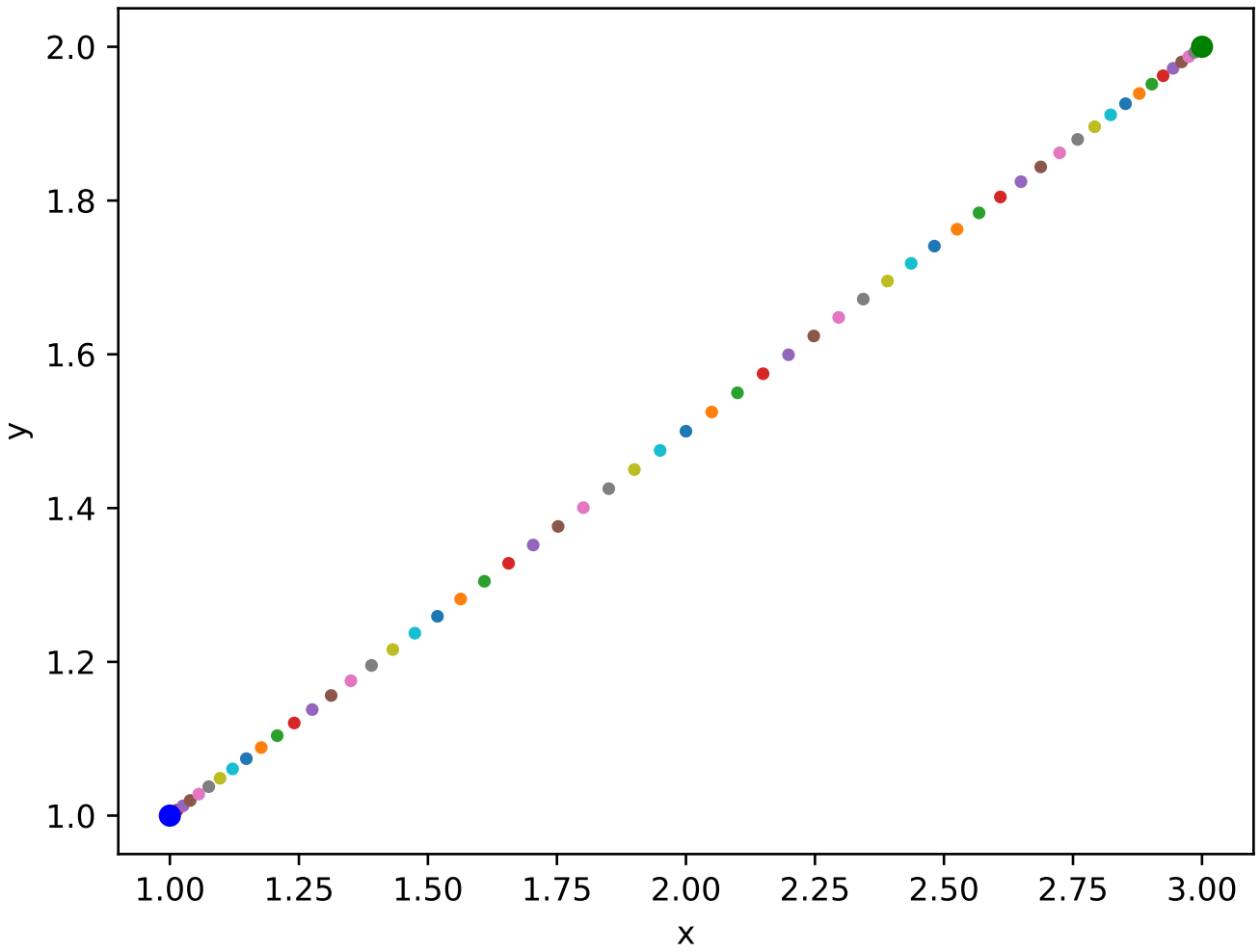
- Even-ordered polynomials are not good! We are always left with overparameterized systems!
- Let's do 3rd order:

$$\boldsymbol{q}(t) = \boldsymbol{c}_3 t^3 + \boldsymbol{c}_2 t^2 + \boldsymbol{c}_1 t + \boldsymbol{c}_0$$

- And:

$$\begin{aligned}
\boldsymbol{c}_0 &= \boldsymbol{q}_s \\
\boldsymbol{c}_1 &= \dot{\boldsymbol{q}}_s \\
\boldsymbol{c}_2 &= \frac{3\boldsymbol{q}_g}{T^2} - \frac{3\boldsymbol{q}_s}{T^2} - \frac{2\dot{\boldsymbol{q}}_s}{T} - \frac{\dot{\boldsymbol{q}}_g}{T} \\
\boldsymbol{c}_3 &= -\frac{2\boldsymbol{q}_g}{T^3} + \frac{2\boldsymbol{q}_s}{T^3} + \frac{\dot{\boldsymbol{q}}_s}{T^2} + \frac{\dot{\boldsymbol{q}}_g}{T^2}
\end{aligned}$$

Splines - Code Example



Bibliography

Chapter 11 from **Modern Robotics: Mechanics, Planning, and Control**, *Kevin M. Lynch and Frank C. Park*, 2017, Cambridge University Press. ebook

Thank you

- **Any Questions?**
- **Office Hours**:
  - **Tue-Thu** *(10:00-12:00)*
  - 24/7 by email (`costashatz@upatras.gr`, subject: *ECE_IR_AM*)
- **Material and Announcements**

Laboratory of Automation & Robotics